KIET DEEMED TO BE UNIVERSITY

SPRING BOOT – CODING ASSESSMENT

Duration: 2 Hours
Maximum Marks: 20
Assessment Type: Practical / Coding

CASE STUDY

KIET Deemed to be University wants to develop a **Student Academic Management REST API** using **Java and Spring Boot** to efficiently manage student academic information. The system will provide a centralized backend service through which authorized applications can access and manage student records. Each student record will contain important details such as **student ID, name, email, branch, year, and CGPA**. The application should provide JSON-based responses so that the REST API can easily communicate with web applications, mobile applications, and other client applications.

The university requires the system to support various operations for managing student records. The REST API should allow users to **retrieve all students, retrieve a particular student using the student ID, add a new student, update existing student information, and delete a student record**. The application should make appropriate use of Spring Boot REST concepts such as `@RestController`, request mappings, `@RequestBody`, `@PathVariable`, and `@RequestParam`. For this assessment, student records can be maintained using an **in-memory `List<Student>`**, and no database integration is required.

In addition to basic CRUD operations, the university wants the API to provide **search and filtering capabilities**. Users should be able to search students based on parameters such as **branch and academic year**, as well as retrieve students whose CGPA meets a specified minimum value. The application should be properly structured as a Spring Boot project and should use appropriate HTTP methods such as **GET, POST, PUT, and DELETE**. Students are expected to implement, run, and test the REST APIs and demonstrate the JSON request and response for the specified operations.

Each student contains:

studentId
name
email
branch
year
cgpa

Example:

```
{
   "studentId": 101,
   "name": "Rahul Sharma",
   "email": "rahul@kiet.edu",
   "branch": "CSE(AI & ML)",
   "year": 2,
   "cgpa": 8.65
}
```

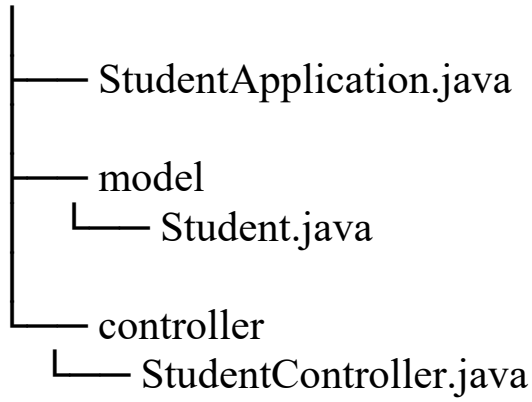
Use an **in-memory List<Student>** for storing student data. No database is required.

QUESTION 1 – Spring Boot Project Setup

3 Marks

Create the Spring Boot application with the following structure:

com.kiet.student



Requirements:

- Create the Spring Boot main class.
- Create the Student model with all required fields.
- Configure the application to run on port 8080.
- Add the required Spring Web dependency.

QUESTION 2 – Runner & Logger

2 Marks

Implement a CommandLineRunner that displays the following messages using a Spring Boot Logger when the application starts:

KIET Student Management System Started
Loading Student Data...
Student Management REST API is Ready

QUESTION 3 – GET REST APIs

4 Marks

Create an in-memory list containing at least **5 students**.

Implement both APIs:

A. Get All Students

GET /students

B. Get Student by ID

GET /students/{studentId}

Use @PathVariable for the second API.

Example:

GET /students/101

If the student does not exist, return:

```
{
   "message": "Student not found"
}
```

QUESTION 4 – POST REST API

3 Marks

Implement:

POST /students

Use @RequestBody.

Accept student data in JSON format:

{

```
    "studentId": 106,
    "name": "Amit Kumar",
    "email": "amit@kiet.edu",
    "branch": "CSE(AI)",
    "year": 2,
    "cgpa": 8.75
}
```

Add the student to the in-memory list and return the newly added student as JSON.

QUESTION 5 – PUT & DELETE REST APIs

4 Marks

Implement both APIs.

A. Update Student

PUT /students/{studentId}

Use:

@PathVariable
@RequestBody

Example:

PUT /students/101

Request:

```
{
    "studentId": 101,
    "name": "Rahul Sharma",
    "email": "rahul@kiet.edu",
    "branch": "CSE(AI & ML)",
    "year": 3,
    "cgpa": 9.05
```

}

B. Delete Student

DELETE /students/{studentId}

Example:

DELETE /students/105

Return:

```
{
   "message": "Student deleted successfully"
}
```

If the student does not exist, return:

```
{
   "message": "Student not found"
}
```

QUESTION 6 – Request Parameter APIs

4 Marks

Implement the following APIs using @RequestParam.

A. Search by Branch and Year

GET /students/search?branch=CSE&year=2

Return all students matching the given branch and year.

B. Find Students Based on CGPA

GET /students/topper?cgpa=9.0

Return students whose CGPA is **greater than or equal to** the supplied CGPA.

Example:

GET /students/topper?cgpa=9.0

MARKING SCHEME

Question	Component	Marks
Q1	Spring Boot Project, Model & Configuration	3
Q2	CommandLineRunner & Logger	2
Q3	GET APIs + PathVariable	4
Q4	POST + RequestBody	3
Q5	PUT + DELETE APIs	4
Q6	RequestParam – Search & CGPA	4
Total		**20**

REQUIRED REST ENDPOINTS

GET /students
GET /students/{studentId}

POST /students

PUT /students/{studentId}
DELETE /students/{studentId}

GET /students/search?branch=CSE&year=2
GET /students/topper?cgpa=9.0

SUBMISSION REQUIREMENTS

Students must submit the following on Moodle

1. Complete Spring Boot source code
2. pom.xml
3. API testing screenshots
4. JSON request/response screenshots

No database is required. Use an in-memory List<Student>.